HDOC Day-68 Activity

Decimal to Binary Conversion Using Recursion

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-68

Aim

To write a recursive C program that converts a non-negative decimal integer into its equivalent **binary representation**.

Concept

Binary numbers use only the digits 0 and 1. To convert a decimal number to binary, repeatedly divide the number by 2 and record the remainder. The remainders must be read in reverse order, which recursion handles naturally.

Problem Requirements

- Read a decimal integer from the user.
- Use a recursive function to convert the decimal value to binary.
- Repeatedly divide the number by 2.
- Print each remainder in the correct binary order.
- Correctly handle decimal 0.

Recursive Idea

For a number n greater than 1, first recursively process **n / 2**. After that recursive call returns, print **n % 2**. Because printing occurs while the recursive calls return, the most significant binary digit appears first.

Algorithm

1. Read a non-negative decimal integer n.
2. If n is 0, directly print 0.
3. Otherwise call **decimal_to_binary(n)**.
4. In the recursive function, if n > 1, recursively call **decimal_to_binary(n / 2)**.
5. Print **n % 2** after the recursive call.
6. The sequence of printed remainders forms the binary representation.

C Program

```
#include <stdio.h>

void decimal_to_binary(unsigned int n)
{
    if (n > 1)
    {
        decimal_to_binary(n / 2);
    }

    printf("%u", n % 2);
}

int main()
{
    unsigned int n;

    printf("Enter a decimal number: ");
    scanf("%u", &n);

    printf("Binary representation: ");

    if (n == 0)
    {
        printf("0");
    }
    else
    {
        decimal_to_binary(n);
    }

    return 0;
}
```

Why Printing Happens After the Recursive Call

During division, the first remainder found is the least significant binary digit. If it were printed immediately, the binary number would appear backwards. Calling the function with n/2 first and printing n%2 afterward reverses the order automatically.

Dry Run - Decimal 13

Call	Division	Remainder	When returning
binary(13)	13 / 2 = 6	1	prints 1 last
binary(6)	6 / 2 = 3	0	prints 0
binary(3)	3 / 2 = 1	1	prints 1
binary(1)	Base level	1	prints 1 first

The digits are printed during the return phase as **1 1 0 1**. Therefore, decimal 13 is binary **1101**.

Sample Test Cases

Decimal	Binary
0	0
1	1

2	10
5	101
10	1010
13	1101
25	11001

Special Case: Decimal 0

The recursive function is designed for positive values. Since the binary representation of decimal 0 is simply **0**, main() handles this case directly.

Complexity

Time Complexity: O(log n), because the decimal number is divided by 2 in every recursive call.

Space Complexity: O(log n), because the recursive call stack contains one level for each division by 2.

Result

The program successfully converts a decimal integer to its binary representation using recursion. Recursive division determines the binary digits, and printing during the return phase places them in the correct order.

Execution Example

```
Enter a decimal number: 13
Binary representation: 1101
```